

At the bottom of the main loop, we draw the ball. Since we're updating the values of `ballX` and `ballY` in every frame, the ball appears to animate smoothly. Try it out. Whenever the ball reaches any edge, it seems to bounce off.

Using Pygame rects

Next I'll present a different way to achieve the same result. Rather than keeping track of the current x- and y-coordinates of the ball in separate variables, we'll use the `rect` of the ball, update the `rect` every frame, and check if performing the update would cause any part of the `rect` to move outside an edge of the window. This results in fewer variables, and because we'll start by making a call to get the `rect` of an image, it will work with images of any size.

When you create a `rect` object, in addition to remembering the `left`, `top`, `width`, and `height` as attributes of the rectangle, that object also calculates and maintains a number of other attributes for you. You can access any of these attributes directly by name using *dot syntax*, as shown in [Table 5-1](#). (I'll provide more detail on this in Chapter 8.)